

基于 Docker 的 Nginx 容器化部署 与版本管理

作业 A6 / 云计算期末项目

第25组 | 2026年1月4日

演示目录

🚩 1. 项目背景与需求目标

🏗️ 2. 系统架构与方案设计

📄 3. 核心实现 (Dockerfile & Script)

🔧 4. 一键复现指南

✅ 5. 功能验证与排错

👥 6. 小组分工与贡献

部署面临的挑战



手动部署的痛点

在传统虚拟机环境中，配置 Nginx 需要重复的手动命令。依赖冲突时有发生，"在我的机器上能跑" 是常见的环境一致性问题。



容器化解决方案

通过将 Nginx 和静态网站打包到 Docker 镜像中，我们保证了运行时环境在所有开发和生产阶段的高度一致性。

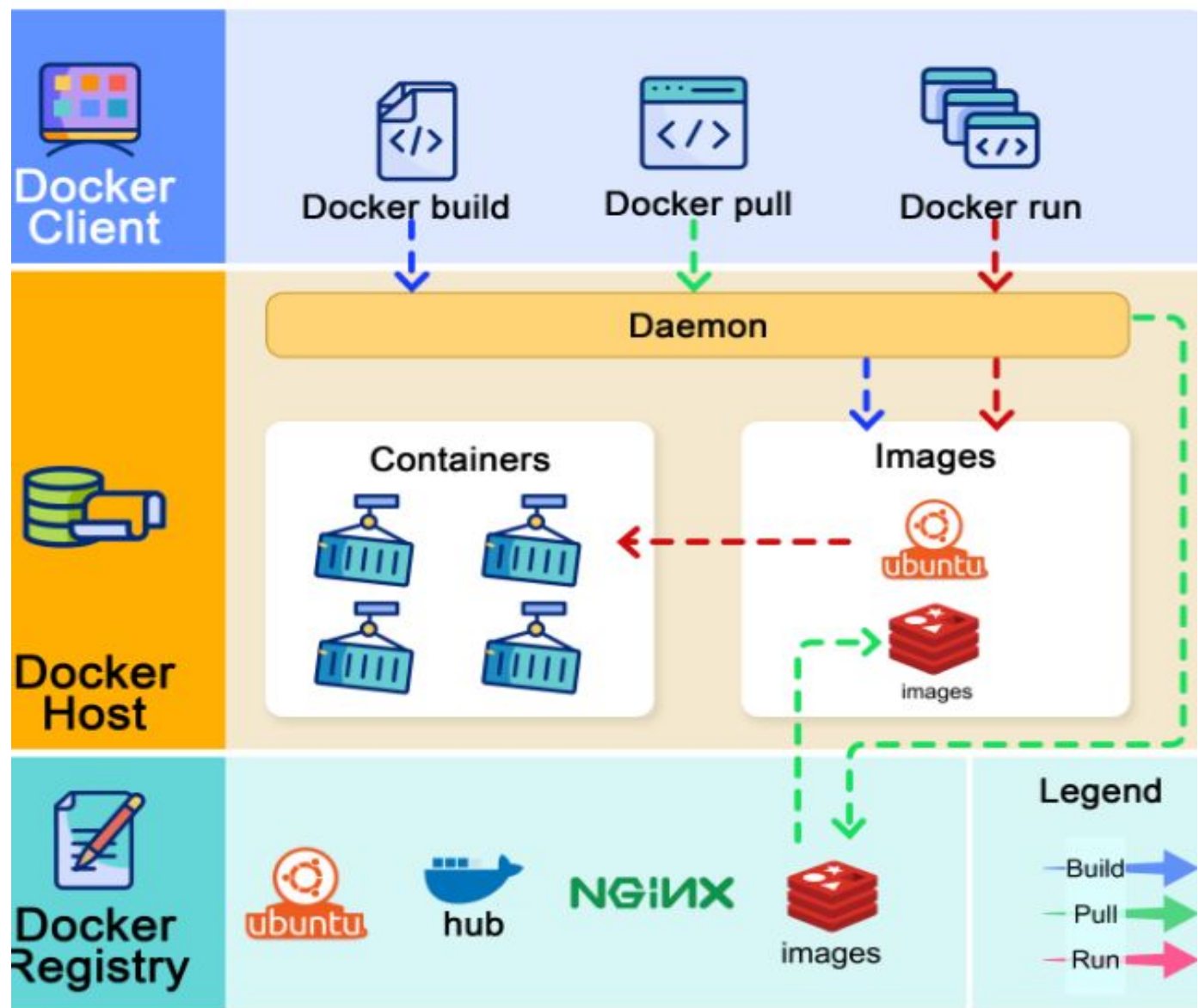
系统架构

我们的架构利用 Docker 的隔离性，同时通过端口映射保持连接。

- 宿主机 (Ubuntu): 管理运行时和存储卷。
- 容器 (Nginx): 在隔离环境中运行 Web 服务器。
- 端口 8080: 映射到内部端口 80 以供访问。

How does Docker Work ?

 blog.bytebytego.com

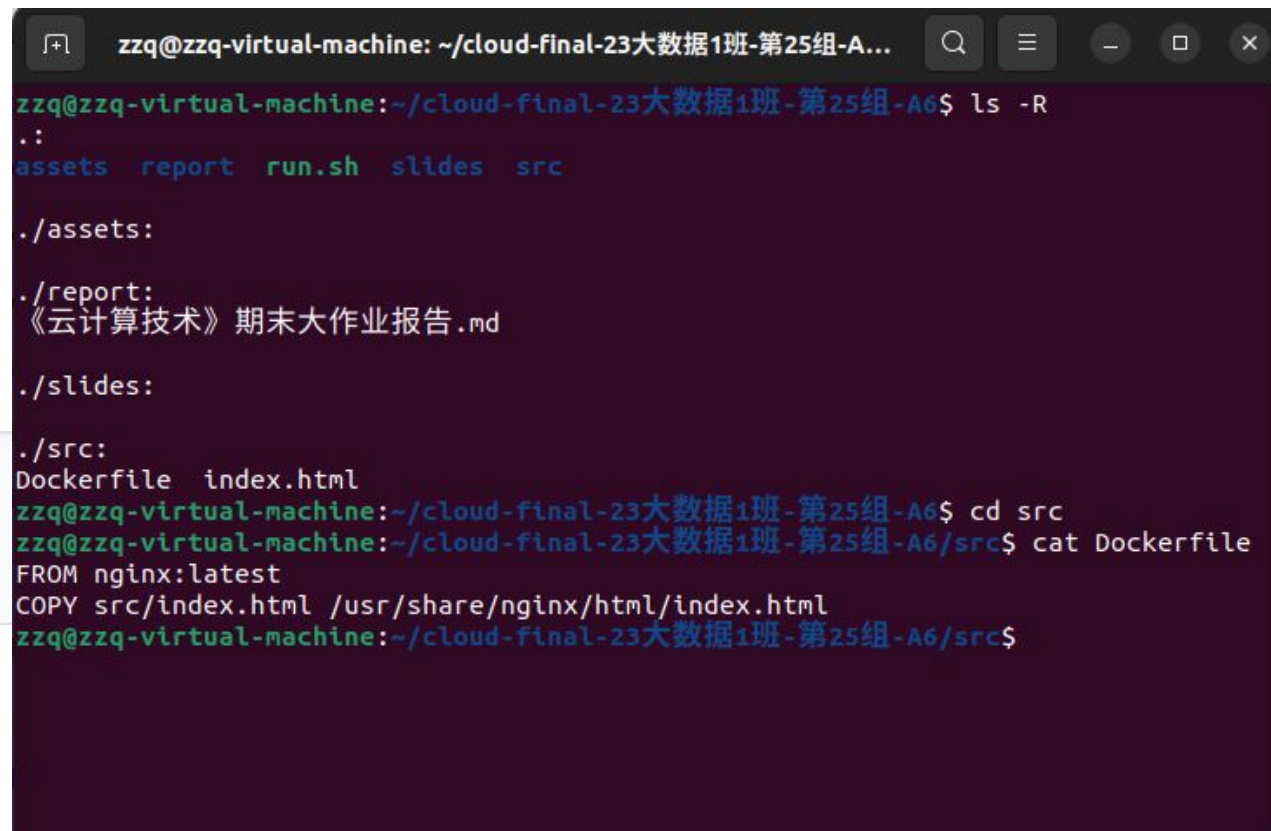


基础设施即代码：Dockerfile

Dockerfile 是我们环境的蓝图。它通过显式定义基础镜像和文件结构来确保标准化。

```
# 使用官方 Nginx 镜像作为基础    FROM nginx:latest
COPY src/index.html /usr/share/nginx/html/index.html
# 将静态文件复制到容器中
```

这种简单的配置消除了在每台服务器上运行 apt-get install nginx 的需求，实现了真正的“一次构建，到处运行”。

A terminal window with a dark background and light-colored text. The window title is 'zzq@zzq-virtual-machine: ~/cloud-final-23大数据1班-第25组-A...'. The terminal shows the following commands and output:
1. Command: `ls -R`
Output: `.:
assets report run.sh slides src`
2. Command: `./assets:`
Output: `./report:
《云计算技术》期末大作业报告.md`
3. Command: `./slides:`
Output: `./src:
Dockerfile index.html`
4. Command: `cd src`
5. Command: `cat Dockerfile`
Output: `FROM nginx:latest
COPY src/index.html /usr/share/nginx/html/index.html`
6. Command: `cat Dockerfile`
Output: `FROM nginx:latest
COPY src/index.html /usr/share/nginx/html/index.html`

自动化：run.sh 脚本

我们将构建和运行逻辑封装在 Shell 脚本中，实现“一键部署”。这防止了输入长 Docker 命令时的人为错误。

1. 清理旧容器，防止冲突

2. 构建带版本标签的镜像

3. 运行容器 (映射端口与挂载)

```
run.sh
~/cloud-final-23大数据1班-第25组-A6
Save

1 #!/bin/bash
2
3 # 1. 接收参数：如果用户没传参数，默认使用 "v1"
4 # $1 代表命令行传入的第一个参数
5 VERSION=${1:-v1}
6
7 IMAGE_NAME="my-nginx:$VERSION"
8 CONTAINER_NAME="my-website"
9
10 echo "=== 开始部署流程 (版本: $VERSION) ==="
11
12 # 2. 清理旧环境
13 # 不管旧容器是 v1 还是 v2，只要名字叫 my-website 就删掉，防止冲突
14 echo "-> [1/3] 清理旧容器..."
15 docker rm -f $CONTAINER_NAME 2>/dev/null || true
16
17 # 3. 构建镜像
18 echo "-> [2/3] 构建镜像 $IMAGE_NAME ..."
19 docker build -t $IMAGE_NAME .
20
21 # 4. 启动容器
22 echo "-> [3/3] 启动容器..."
23 docker run -d \
24   -p 8080:80 \
25   -v "$(pwd)/src":/usr/share/nginx/html \
26   -e STUDENT_NAME="Group25" \
27   --name $CONTAINER_NAME \
28   $IMAGE_NAME
29
30 echo "=== 部署完成！ ==="
31 echo "当前运行版本: $IMAGE_NAME"
32 echo "验证地址: http://localhost:8080"
```

深入解析：运行时配置



端口映射

`-p 8080:80`

将容器内部的 Web 服务器端口 (80) 暴露给主机的端口 8080，使其可通过 localhost 访问。



数据持久化

`-v $(pwd)/src:...`

将本地源目录挂载到容器中。HTML 文件的更改会立即反映（热重载），方便开发调试。



环境变量

`-e`

`STUDENT_NAME...`

在运行时动态注入配置数据，而无需更改镜像代码，提高了镜像的通用性。

| 一键复现指南

我们已将所有代码开源，您仅需三步即可在任何 Linux Docker 环境下完美复现本实验：

步骤 1: 克隆代码仓库

步骤 2: 进入项目目录

步骤 3: 运行自动化脚本 (默认 v1 版本)

进阶： 运行 `./run.sh v2` 可切换至 v2 版本。

```
git clone  
[https://github.com/ZZQ-77756/cloud-final-class1-group25-A6.git](https://github.com/ZZQ-77756/cloud-final-class1-group25-A6.git)
```

```
cd cloud-final-class1-group25-A6
```

```
./run.sh
```

敏捷版本管理



版本 1 (v1)

命令: `./run.sh v1`

部署基准静态网站。主要用于验证 Dockerfile 构建过程是否正常，以及端口映射是否生效。



版本 2 (v2)

命令: `./run.sh v2`

部署更新后的内容。演示脚本如何无缝处理容器替换和标签管理，展示了容器发布的灵活性。

版本v2

```
zzq@zzq-virtual-machine: ~/cloud-final-23大数据1班-第25组-A6
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ echo '<h2>Hello from Version 2!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!</h2>' >> src/index.html
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ ./run.sh v2
=== 开始部署流程 (版本: v2) ===
-> [1/3] 清理旧容器...
my-website
-> [2/3] 构建镜像 my-nginx:v2 ...
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

        Install the buildx component to build images with BuildKit:
        https://docs.docker.com/go/buildx/

unable to prepare context: unable to evaluate symlinks in Dockerfile path: lstat
/home/zzq/cloud-final-23大数据1班-第25组-A6/Dockerfile: no such file or directory
-> [3/3] 启动容器...
7a851df88d0b7e335fe51e090ab779071ae4578a6f5450d3a787d231af74293f
=== 部署完成! ===
当前运行版本: my-nginx:v2
验证地址: http://localhost:8080
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ curl localhost:8080
<h1>这是第25组的云计算大作业</h1><p>基于 Nginx 的静态网页实验</p>
<p>Volume Mounting Test Success! (Updated from Host)</p>
<h2>Hello from Version 2!</h2>
<h2>Hello from Version 2!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!</h2>
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$
```

结果验证

成功标准达成：

- 网站可通过 localhost:8080 正常访问。
- 页面显示 "Group 25" (验证环境变量注入成功)。
- 版本切换时，网页内容即时更新，无服务中断感。

```
zzq@zzq-virtual-machine: ~/cloud-final-23大数据1班-第25组-A6
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ ./run.sh v1
=== 开始部署流程 (版本: v1) ===
-> [1/3] 清理旧容器...
my-website
-> [2/3] 构建镜像 my-nginx:v1 ...
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

          Install the buildx component to build images with BuildKit:
          https://docs.docker.com/go/buildx/

unable to prepare context: unable to evaluate symlinks in Dockerfile path: lstat
/home/zzq/cloud-final-23大数据1班-第25组-A6/Dockerfile: no such file or directory
-> [3/3] 启动容器...
120cbb984f121019c18d9d45cdf782548ce44552b213d34784ae2f9a79847d42
=== 部署完成！ ===
当前运行版本: my-nginx:v1
验证地址: http://localhost:8080
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ curl localhost:8080
<h1>这是第25组的云计算大作业</h1><p>基于 Nginx 的静态网页实验</p>
<p>Volume Mounting Test Success! (Updated from Host)</p>
<h2>Hello from Version 2!</h2>
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$ docker inspect my-website | grep STUDENT_NAME
          "STUDENT_NAME=Group25",
zzq@zzq-virtual-machine:~/cloud-final-23大数据1班-第25组-A6$
```

遇到的挑战与解决方案



Alpine 镜像网络超时

我们尝试使用 `nginx:alpine` 以减小镜像体积。然而，由于国内网络环境对 Docker Hub 的连接限制，构建过程在拉取层时超时。



解决方案与复盘

我们回退到了本地已缓存的 `nginx:latest`。这教会了我们为生产环境配置国内镜像加速器或搭建私有仓库 Harbor 的必要性。

小组分工与贡献



成员 1

职责：环境与脚本

负责 Ubuntu 虚拟机基础环境搭建，Docker 服务配置；编写核心自动化脚本 run.sh，实现一键部署逻辑。



成员 2

职责：镜像与验证

编写与优化 Dockerfile；负责镜像的构建测试，执行 v1/v2 版本切换的功能验证，记录实验数据。



成员 3

职责：文档与展示

汇总实验数据撰写实验报告；整理排错记录 (Troubleshooting)；制作并美化答辩 PPT。

项目总结与收获

- ★ **生命周期管理：** 深刻理解了容器从构建、运行到销毁的完整生命周期。
- ★ **基础设施即代码：** 掌握了通过 Dockerfile 和 Shell 脚本实现环境标准化的能力。
- ★ **数据持久化：** 厘清了容器内部存储与宿主机挂载卷 (Volume) 之间的关键区别。

项目资源



源代码仓库

访问完整的项目代码，包括 run.sh 脚本、Dockerfile 配置以及详细的 README 文档。

 [github.com/ZZQ-77756/cloud-final-class1-group25-](https://github.com/ZZQ-77756/cloud-final-class1-group25-A6)
A6

问答环节

感谢您的观看

第25组 | 云计算课程